Repair ledger — Aletheia Evidence Bundle

Issues found and fixed during the build, in Project Witness's "make claims, attack
claims, repair claims, narrow claims" spirit. Newest first.

R1 — FIRST BUILD (v0.1.0): Aletheia Evidence Bundle

First Aletheia build — Move 9 of Project Witness, and the first piece of Aletheia.
It bundles ONE local file as evidence: hash it, record metadata in a canonical
manifest, write a strict checksums file, copy the bytes into
`evidence/evidence_001.bin`, and verify the bundle's integrity later. It is not
connected to the workflow yet.

Scope held deliberately narrow: one local file only; no directories, web,
sensors, OCR, image analysis, LLM, truth claims, signing, or anchoring. The

component proves preservation/hash integrity only — `VERIFIED_PRESENT` is an integrity verdict, never a truth verdict.

Design decisions worth recording:

- **Honest status vocabulary.** The status set is small and integrity-scoped (`VERIFIED_PRESENT` plus five `FAILED_*` reasons). No `PASS` / `TRUE` / `AUTHENTIC` value exists; a regression test asserts no status string implies truth, and the manifest's `known_limitations` explicitly disclaims proving contents true.
- **Build-time integrity invariant.** The copied evidence file's hash must equal the source file's hash, or the build is refused (`COPY_HASH_MISMATCH`). The manifest records both, and verify re-checks `source_file_sha256 == bundled_file_sha256`.
- **Strict source validation.** Missing / directory / symlink / oversized sources are refused. Symlinks are detected with `os.lstat` BEFORE following, so a symlink to a valid file is still refused (`SYMLINK_SOURCE`). The 10 MB v0.1 limit is enforced and documented.
- **Strict checksums format.** Exactly two `sha256sum`-style lines; duplicate,

missing, unknown keys and malformed digests all fail (`FAILED_CHECKSUMS_INVALID`). Uppercase digests are rejected (canonical lowercase only).

- **Canonical manifest enforced on verify.** The manifest must be byte-for-byte canonical; a pretty-printed or reordered manifest fails with the specific `FAILED_NON_CANONICAL_MANIFEST` status, distinct from structural invalidity.
- **Unknown-file rejection.** verify refuses any unknown file at the top level or inside `evidence/`, so a bundle is a closed set of known files.
- **Reused the stack's honest patterns:** deterministic canonical JSON; atomic writes (`os.replace`); refuse a non-empty output directory; `require()` not bare assert; in-process selfcheck that never launches the unit suite.

Release gate (Coordinator v0.1.8 / Companion v0.1 lesson applied directly):
there is NO subprocess orchestrator in the freeze gate. The authoritative gate is the explicit command list in RELEASE_COMMANDS.md plus the static `verify_release_manifest.py` (which launches no

subprocesses and additionally
checks that the README scopes the tool to integrity,
not truth). The manual smoke
test is documented but not part of the gate.

Verified: all release-gate commands pass individually
from a clean extraction
(see TEST_RESULTS.md); selfcheck 8/8 in-process
gates; verify_release_manifest.py
passes (7/7, no subprocesses); the manual smoke
test produces
`evidence_status = VERIFIED_PRESENT` and verify
confirms it; tamper to the evidence
file, the manifest, the checksums, or the file set is
detected with the correct
`FAILED_*` status.